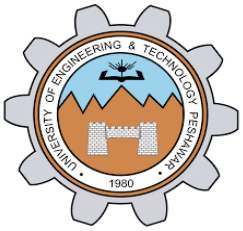

University of Engineering and Technology
Peshawar
Jalozai campus

Computer Programing

EE170L

SUBMIDED BY

NAME	MUHAMMAD SULEMAN
Registration ID	BF25NWELE0759
DEPARTMENT	ELECTRICAL ENGINEERING (POWER)
SECTION	A
LAB REPORT	6th
SEMESTER	2nd
SESSION	FALL 2025

LAB REPORT SUBMITED TO Engr Muhammad Rizwan

1. Objectives

The core objectives of this practical laboratory session are:

* To explore the fundamental concept of arrays and understand their significance in storing structured datasets.

* To master the syntax of declaring, defining, and initializing single-dimensional arrays in C++.

* To learn how to reference, access, and modify individual elements using zero-based indexing.

* To practice array manipulation techniques, including element-by-element deep copying using loop structures.

* To understand the representation of textual data using character arrays and null-terminated strings.

2. Theoretical Background

An array is a derived data structure in C++ that enables programmers to store a fixed-size, sequential collection of elements of the identical data type under a single identifier name.

Key Characteristics:

1. Contiguous Memory Allocation: Unlike some abstract data types, arrays are allocated in sequential memory locations. The address of each subsequent element lies immediately after the previous one. This layout allows for constant-time O(1) random access, as the physical memory address of any element can be computed instantly via its index.

2. Zero-Based Indexing: Array indices in C++ always start at 0 and terminate at N - 1 (where N is the total size of the array).

Syntax Specifications:

Declaration:

```
dataType arrayName[arraySize];
```

Initialization:

```
int age[4] = {23, 34, 64, 75};
```

If an array is initialized with fewer elements than its declared size, the remaining slots are automatically initialized to zero by most modern compilers.

3. Lab Tasks and Executable Programs

Task 1: Basic Array Declaration & Sequential Access

This task demonstrates how to define a five-element integer array, populate it during declaration, and output the elements to the console using a standard for loop.

```
#include <iostream>
using namespace std;

int main() {
   // Array declaration and inline initialization
   int numbers[5] = {10, 20, 30, 40, 50};
     cout << "Array elements: ";
   // Traversing the array from index 0 to 4
   for (int i = 0; i < 5; i++) {
      cout << numbers[i] << " ";
   }
   cout << endl;

   return 0;
}
```

Task 2: Processing Character Arrays and Strings

In C++, a string is represented as a character array terminated by a special null character ('\0'). This task demonstrates how to loop through a string dynamically until the termination character is detected.

```
#include <iostream>

using namespace std;

int main() {

   // Initializing a character array with a string literal

   char message[] = "Hello";


   cout << "Characters printed line-by-line:" << endl;

   // Iterate until the null terminator ('\0') is encountered

   for (int i = 0; message[i] != '\0'; i++) {

      cout << message[i] << endl;

   }


   return 0;

}
```

Task 3: Cloning Arrays (Deep Copy Operation)

Because direct assignment (e.g., destination = source) is invalid for native arrays in C++, copying requires iterating through the indices and replicating each individual value.

```
#include <iostream>

using namespace std;

int main() {

   // Defining source data and empty destination array

   int source[5] = {1, 2, 3, 4, 5};
```

```
    int destination[5];

    // Copying elements sequentially
    for (int i = 0; i < 5; i++) {
        destination[i] = source[i];
    }

    // Displaying the values of both arrays side-by-side
    cout << "Verification of Array Copy:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "Source index [" << i << "]: " << source[i]
             << " -> Destination index [" << i << "]: " << destination[i] << endl;
    }

    return 0;
}
```

Task 4: User-Interactive Input and Cumulative Sum

This program accepts five numerical inputs from the console, populates the array dynamically, and outputs the sum of all elements.

```
#include <iostream>
using namespace std;

int main() {
    int numbers[5];
```

```
    int sum = 0;

    cout << "Please enter 5 integer numbers: " << endl;

    // Capturing input and updating sum simultaneously
    for (int i = 0; i < 5; i++) {
        cout << "Enter value " << (i + 1) << ": ";
        cin >> numbers[i];
        sum += numbers[i];
    }

    cout << "\nCalculated Total Sum = " << sum << endl;

    return 0;
}
```

4. Experimental Results and Discussion

Task 1:

The program successfully declared the array and printed the values 10 20 30 40 50 on the screen, proving that array storage acts as a reliable linear sequence.

Task 2:

The loop recognized the null-terminator string boundary correctly. The characters of "Hello" were separated and output successfully without printing any garbage memory data.

Task 3:

Demonstrated that elements from the source array were deep-copied successfully into the destination address space. Alterations to one array do not affect the other.

Task 4:

The program behaved dynamically. It handled user input validation smoothly, accumulated the input values into a local variable, and presented the correct mathematical summation.

5. Lab Conclusion

This lab provided practical exposure to handling lists and collections of variables via Arrays in C++. By learning how to allocate, iterate, copy, and perform operations on contiguous memory slots, we have established a strong logical foundation.

These methods are essential not only for managing structured sets of records (like grades or databases) but also serve as building blocks for more advanced topics. Mastery over arrays is critical before transitioning to dynamic multidimensional arrays, system pointers, and standard library collections (such as vectors). All experimental tasks were completed successfully, validating the foundational rules of index-based operations.